CASE STUDY REPORT

13. Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

NAME : VAIDEESHWARI

REGISTER NO.: 241401121

241401121

Table of Contents

1. Abstract
2. Introduction
3. Background: Understanding Low-Code
4. Case Scenario: Modernising a Global Freight Platform
5. Limitations Encountered
6. Impact Analysis
7. Comparison: Low-Code vs. Traditional Development
8. Lessons Learned and Recommendations
9. Where Low-Code Remains the Right Choice
10. Conclusion
11. References

24140121

1. Abstract

Low-code development platforms (LCDPs) such as OutSystems, Mendix, Microsoft Power Apps and Appian aim to shorten the path from idea to working software by replacing most hand-written code with visual models, ready-made components and managed infrastructure. For internal tools, approval workflows and prototypes this approach works very well. The picture changes when the same platforms are asked to carry large, mission-critical enterprise systems. This report studies an illustrative scenario in which a global freight company tries to rebuild its shipment and customs platform on a low-code product. It records the technical, architectural, financial and organisational limits the project ran into, assesses what they cost, and sets out guidance on when and how an enterprise should adopt low-code.

2. Introduction

Organisations are under steady pressure to release digital products quickly at a time when experienced developers are hard to hire. Market analysts have for years predicted that low-code and no-code tooling will make up a rising proportion of new applications. The attraction is easy to understand: faster releases, smaller upfront spending, and the chance to let business staff (often called citizen developers) build their own solutions.

However, a fast start is not the same as a sustainable result. Complex enterprise systems combine heavy transaction loads, dense business rules, older back-end systems, tight regulation and lifespans measured in decades. Each of these puts strain on the abstraction that a low-code platform provides. The aim of this report is to examine, through a worked scenario, the points at which that abstraction stops helping and starts to hold a project back.

3. Background: Understanding Low-Code

A low-code platform lets people assemble applications in a visual workspace, using drag-and-drop screens, model-driven process design, configurable connectors and reusable templates. Hand-written code can normally be inserted where the visual tools fall short, but the runtime engine, the data layer and the release pipeline remain largely in the vendor's hands.

The usual advantages are quick prototyping, built-in login and access control, simple deployment and a smaller need for specialist programmers. The very same design choices that deliver these advantages, namely heavy abstraction and vendor-run infrastructure, are also the root of the problems that appear when a system grows in size and complexity.

4. Case Scenario: Modernising a Global Freight Platform

4.1 The Organisation and Its Goal

MeridianFreight Logistics (a fictional name used only for illustration) works in 22 countries and handles about four million shipments each year. Its transport management system had grown over 18 years from COBOL batch jobs and Java services, and every change was slow and risky. Senior management chose a well-known low-code platform to rebuild shipment booking, customs documentation, route planning, billing and customer notifications within 15 months, aiming to cut delivery time and cost by 35% against a traditional rewrite.

4.2 Factors That Made the System Complex

- **Load:** holiday-season peaks of more than 80,000 shipment tracking events per minute.
- **Rules:** thousands of country-specific tariff, duty and customs-classification rules that change several times a year.
- **Integration:** over 25 connected systems, including carrier APIs, EDI gateways, port community systems, vehicle telematics feeds and the corporate ERP used for billing.
- **Compliance:** customs regulations, GDPR, audit trails and rules on where data may be stored.
- **Lifespan:** an intended working life of 12 years or more, with constant modification.

4.3 How the Project Unfolded

The customer booking portal and basic shipment entry went live in three months, well inside the schedule, and stakeholders were delighted. Once work reached customs rule evaluation, carrier integration and high-volume event processing, however, progress dropped sharply. By month 13 the team had written large amounts of custom code and stood up several external microservices simply to get around platform restrictions. The programme was finally reorganised as a hybrid design in which low-code was kept only for the user interface and workflow layers. Final spending ended above what a conventional build had been estimated to cost.

5. Limitations Encountered

5.1 Scalability and Performance

During load tests the platform's generated runtime could not keep up with the event volumes the business needed. The database queries it produced were not efficient, and the engineers had little room to adjust indexing, caching or execution plans. Capacity was tied to the vendor's pricing tiers and hosting rather than to design decisions the team could make. Problems that a conventional team would fix by refactoring turned into vendor support tickets and chargeable upgrades.

24140121

5.2 Managing Complex Business Logic

Visual rule editors are comfortable for straight-line workflows but become hard to follow once conditions nest deeply. The customs and tariff rules, with their thousands of branches, produced enormous diagrams that were difficult to read, review, test and version. One engineer described them as "flowcharts that no one can follow". Habits such as automated unit tests and peer code review were hard to carry over to visual models.

5.3 Integration with Legacy and External Systems

Ready-made connectors covered popular SaaS tools, yet the mainframe billing interface and the carrier EDI feeds needed purpose-built adapters. Batch file exchange, message queues and unusual authentication schemes all called for custom extensions in the platform's own scripting language. Those extensions were thinly documented and bound tightly to a specific platform version, which made the integrations brittle whenever the vendor released an update.

5.4 Vendor Lock-In

Applications were stored in a proprietary model format and executed on the vendor's runtime. Any code that could be exported was generally unreadable and not portable. Moving away would therefore have meant rebuilding almost everything, so switching costs were very high. This reduced MeridianFreight's bargaining power at contract renewal and tied its technical direction to the vendor's product choices.

5.5 Limited Customisation and Flexibility

Needs that fell outside the component library required workarounds. Unusual interface behaviour, specialist routing algorithms and tight control over concurrency and memory were hard or impossible to achieve. Every workaround added technical debt and undermined the original reason for choosing low-code.

5.6 Security, Compliance and Data Residency

The platform supplied a baseline of security features, but the company could not examine the underlying code or hosting environment itself. Auditors requested proof on encryption practice, patch timing and data location, and the answers rested on vendor paperwork. Meeting some national data-storage rules meant buying expensive private-cloud options. Access rules set up incorrectly by inexperienced citizen developers added further governance risk.

5.7 Governance, Shadow IT and Application Sprawl

Because building an app was so easy, different business units created overlapping tools with no central oversight. Duplicated data models, uneven security settings and unmanaged dependencies built up across the portfolio. Without firm standards and clear ownership, the cost of maintenance rose steadily.

5.8 Cost at Scale

Licence fees are usually based on users, applications or consumed resources. Figures that looked small during the pilot climbed quickly as external customers, data volumes and extra environments were added. A total-cost-of-ownership review at month 11 showed that licences, specialist contractors and workaround development had consumed much of the expected saving.

5.9 Skills, Testing and DevOps Maturity

Developers experienced in this particular platform were scarce and costly. Linking the platform to the company's CI/CD pipeline, automated regression suites and source-control branching was more restricted than with ordinary tools. The idea that low-code removes the need for developers did not survive contact with complex work; professional engineering expertise was still required throughout.

5.10 Summary of Limitations

Limitation	Symptom Observed in the Case	Severity
Scalability	Event throughput fell short during peak season	High
Complex logic	Unreadable rule models and weak testing	High
Legacy integration	Custom adapters and brittle interfaces	High
Vendor lock-in	Expensive exit and lost pricing leverage	High
Customisation	Workarounds and growing technical debt	Medium
Security / compliance	Audit evidence depended on the vendor	Medium
Governance	App sprawl and inconsistent standards	Medium
Cost at scale	Licence growth beyond the forecast	High
Skills / DevOps	Scarce talent and pipeline restrictions	Medium

6. Impact Analysis

These limitations had real and measurable effects on the programme. The table below gives an illustrative comparison between what was planned and what was eventually achieved.

Metric	Planned	Realised (Illustrative)
Delivery time	15 months	25 months
Cost versus a conventional rewrite	35% lower	About 20% higher
Peak events supported (per minute)	80,000	About 30,000 before redesign
Share of system built on low-code	100%	About 40% (interface and workflow)
Custom code and external services	Minimal	Substantial (core rules and integration)

Beyond the numbers, the company saw weaker stakeholder confidence, missed customs-compliance commitments and tired teams worn down by repeated redesign. The figures are invented for illustration; their purpose is to show how the limitations reinforce one another, not to report data from an actual organisation.

7. Comparison: Low-Code vs. Traditional Development

Aspect	Low-Code Platform	Traditional Development
Early progress	Very quick for standard applications	Slower to get started
Complex logic	Awkward to manage visually	Flexible and testable in code
Control over scaling	Bounded by the vendor runtime	Complete architectural control
Integration depth	Strong only for common connectors	Unrestricted with custom code
Portability	Low because of lock-in	High through open standards
Cost pattern	Cheap at first, licences keep rising	Higher at first, more predictable
Skills needed	Platform-specific	General software engineering
Best suited to	Workflows, forms, prototypes	Core systems, high performance

8. Lessons Learned and Recommendations

- **Test fit before committing.** Compare expected transaction volume, rule complexity and integration needs with what the platform can do, using a proof of concept that attacks the hardest requirements first rather than the easiest.
- **Choose a hybrid architecture.** Apply low-code to interfaces, workflows and fast-changing layers, and leave performance-critical, rule-heavy cores in conventional, well-tested services exposed through APIs.
- **Plan the exit from day one.** Keep business logic and data models as portable as possible, document them outside the platform, and secure export rights for data and code in the contract.
- **Calculate total cost of ownership.** Count licences at the size you expect to reach, extra environments, contractors and workaround development across a five-to-ten-year period.
- **Set up governance early.** Form a centre of excellence, agree naming and security standards, require approval for new applications and name an owner for each.
- **Keep engineering discipline.** Use version control, automated tests, code review and CI/CD on low-code work wherever the platform allows.
- **Confirm security and compliance proof.** Collect vendor certifications, penetration-test reports and data-location guarantees before design choices become final.

9. Where Low-Code Remains the Right Choice

None of the above makes low-code unsuitable for large organisations. It is still an excellent fit for internal approval flows, data-collection forms, dashboards, customer self-service portals, process automation, proof-of-concept builds, and applications that are small in scale or short-lived. What matters is matching the tool to the job. The error in this case was not selecting low-code; it was expecting low-code to bear the heaviest core workload in the company.

10. Conclusion

Low-code platforms offer genuine benefits by speeding up delivery and bringing more people into software creation, but their abstraction has a cost. In complex enterprise settings marked by large scale, dense logic, deep legacy integration and strict compliance, weaknesses in scalability, flexibility, portability, cost control and governance can cancel out the early gain in speed. The MeridianFreight scenario suggests that the most dependable route is a deliberate hybrid strategy: use low-code where it is strong, keep conventional engineering where control is essential, and plan for governance and exit from the very beginning. Organisations that treat low-code as one tool among several, instead of a replacement for software engineering, are the ones most likely to gain from it.

11. References

- Gartner, Inc. Research publications on enterprise low-code application platforms (Magic Quadrant series).
- Forrester Research. Reports on low-code development platforms and the future of application development.
- Bock, A. C., and Frank, U. (2021). Low-Code Platform. Business & Information Systems Engineering, 63(6), 733-740.
- Sahay, A., Indamutsa, A., Di Ruscio, D., and Pierantonio, A. (2020). Supporting the understanding and comparison of low-code development platforms. 46th Euromicro Conference on Software Engineering and Advanced Applications.
- Vendor documentation: OutSystems, Mendix, Microsoft Power Apps and Appian official platform documentation.

24140121